

Week 2 - Python Basics - Tutorial

Before You Start

Please take some time to familiarise yourself with the Jupyter Notebook

The Jupyter Notebook documentation: <https://docs.jupyter.org/en/latest/>

Before starting to code, please read through this brief to get an overview of the task at hand. You are not expected to complete the entire brief within the allotted two hours, but to make a start and continue outside of the class.

Attempt to complete this set of tasks before next week. Any issues seek help from the lecturers and tutors.

Introduction

The learning aims of this session are to familiarise you with the Jupyter Notebook environment and enable you to write simple Python modules using the `input()` and `print()` functions.

Launching JupyterLab

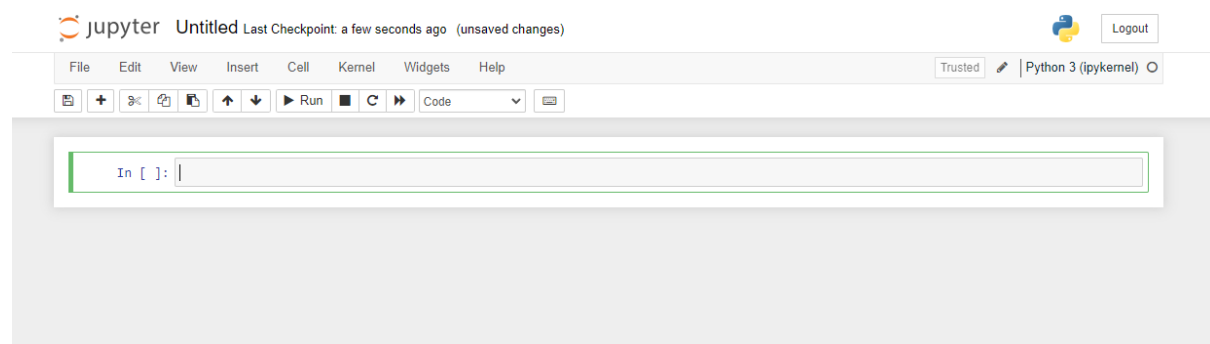
- **Linux** (MATE desktop) go to the Applications top menu | scroll to Programming | click Anaconda Navigator (Anaconda3).
- **Windows** press the Windows key for the start menu | scroll to Anaconda3 (64 bit) folder | and click the Anaconda Navigator (Anaconda3) | and then launch the Jupyter Notebook application

Open Notebook

When you launch the Jupyter Notebook, you will see the below screen.

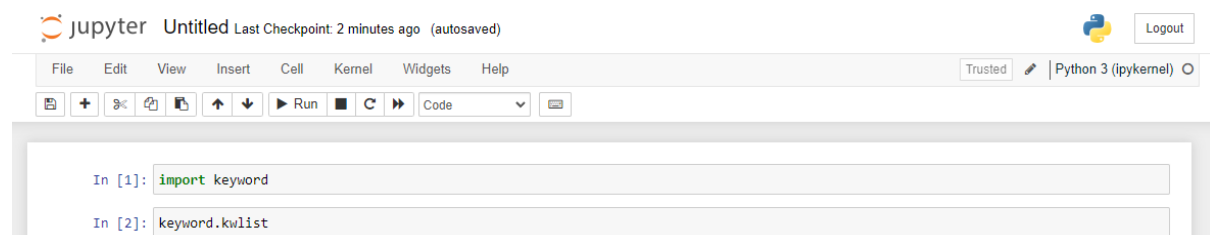


In the upper-right, select New and click on Python3. It will open Untitled.ipynb notebook and you will see the below screen.



Jupyter Notebook Interactive

The Jupyter Notebook prompt is cell Type the following commands in cell . For each new cell click on Insert.



These are Python's reserved words (you cannot use these as identifiers or variable names).

Now type the following at the cell:

```
Hello
```

Note the red error message.

There are several pieces of information given to help identify the error including the line number and the nature of the error.

Always try to understand what the error message is trying to convey so that you can correct your code.

What was the problem in this case?

Now type the following at the cell:

```
print(Hello)
```

Why did that not work?

Now type the following at the cell:

```
print("Hello")
```

Why did that work?

Now type the following at the cells, note the responses and discuss why.

```
type(print)
```

```
type(Hello)
```

```
type("Hello")
```

Jupyter Notebook (Types)

Data structures are used to store data for programs to use. Some simple structures are illustrated below.

Type the following at the Jupyter Notebook cells:

```
v = 10  
  
w = 2.7  
  
x = "Hello"  
  
y = chr(97)
```

Now predict the outcome from each of the following type statements before executing:

```
type(v)  
  
type(w)  
  
type(x)  
  
type(y)
```

What is the output if you type the following?

```
print(y)
```

Make sure you understand why the output results from each case (ask tutors).

Jupyter Notebook (Operators)

What do you think will happen in each of the following cases if you type the following at the cell and then try it (type each in a different cell):

<pre>x = 11 x = x + 1 print(x)</pre>	<pre>y = 11 y = y + 1</pre>	<pre>z = "23" z = "23" + "4" print(z)</pre>
<pre>x = 2 y = 3 z = y / x print(z)</pre>	<pre>x = 2 y = 3 z = y / 2 print(z)</pre>	<pre>x = 7 y = x * x print(y)</pre>

<pre>x = 4 y = x ** x print(y)</pre>		
--------------------------------------	--	--

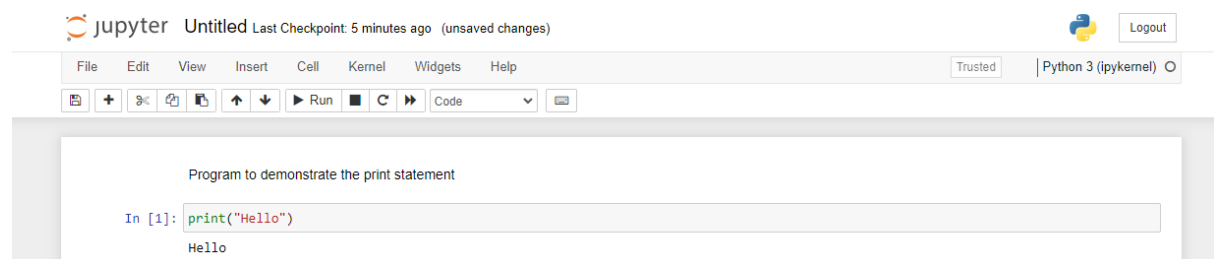
Notebook: program_1.ipynb

A notebook is simply storing your program code.

To create a new notebook select File | New Notebook | Python3.

This will open a notebook editor window into which you can type your program statements.

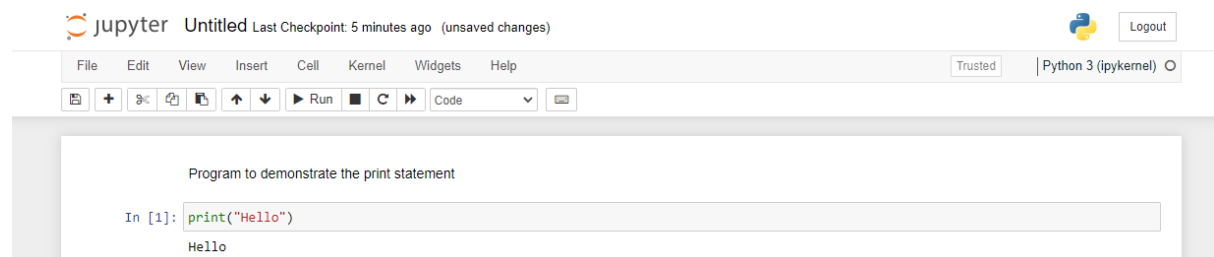
Type the following into the text editor (to add the titles, i.e. non-coding lines, select Cell | Cell Type | Markdown and run the cell):



Save your notebook (program) by selecting File | Save As.

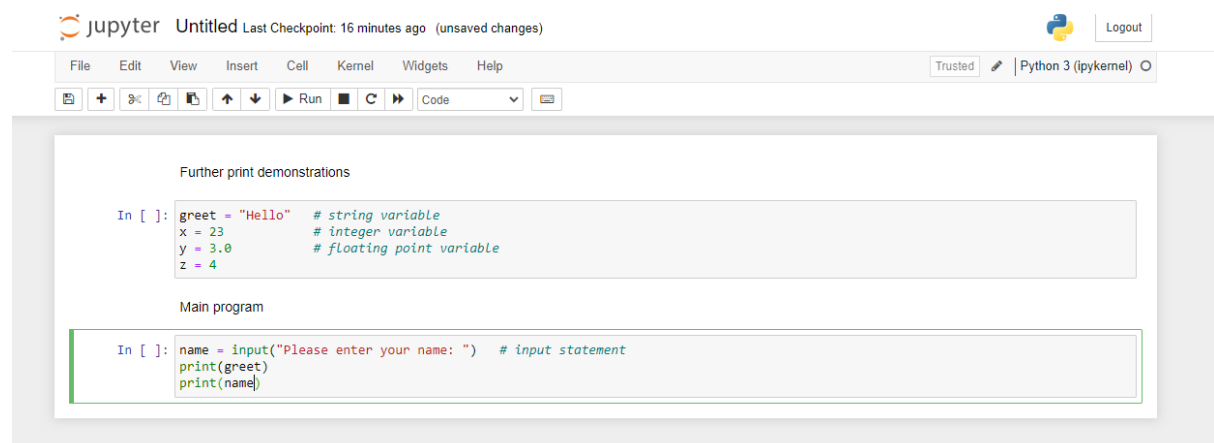
Run your Notebook (program) by selecting Kernel | Restart & Run All

Your notebook should now run and display the following output.

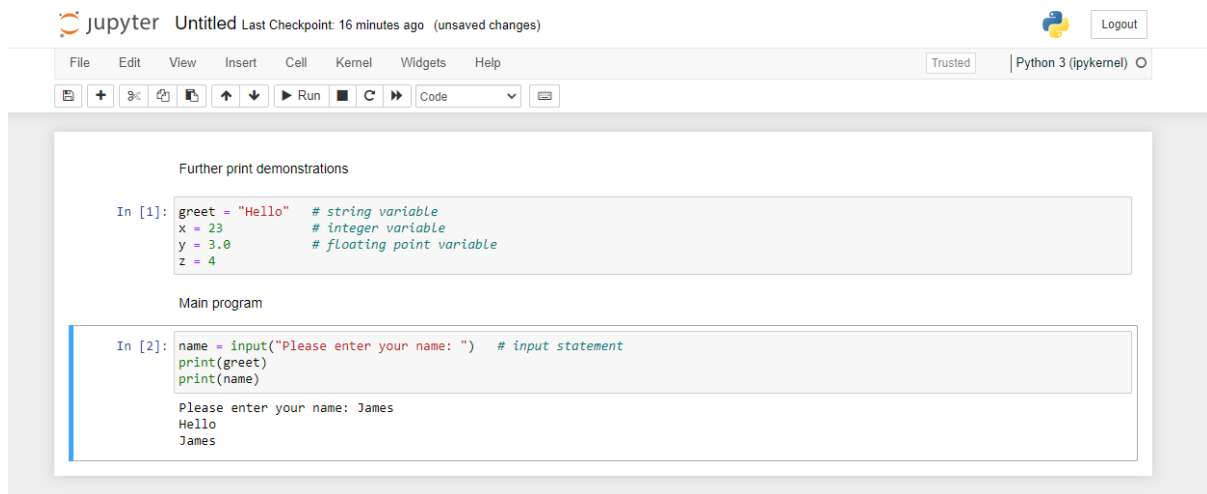


Notebook Practice: program_2.ipynb

Create a new notebook and type the following:



Now save and run your module. It should produce the output below.



```
Further print demonstrations

In [1]: greet = "Hello" # string variable
        x = 23         # integer variable
        y = 3.0        # floating point variable
        z = 4

Main program

In [2]: name = input("Please enter your name: ") # input statement
        print(greet)
        print(name)

Please enter your name: James
Hello
James
```

How can we modify the program to print the greeting and your name on the *same* line?

Modify the final two print statements too:

```
print(greet, name)
```

Now save and run your notebook. Check the greeting and your name is on the same line.

You can also achieve this output using the `end=""` argument:

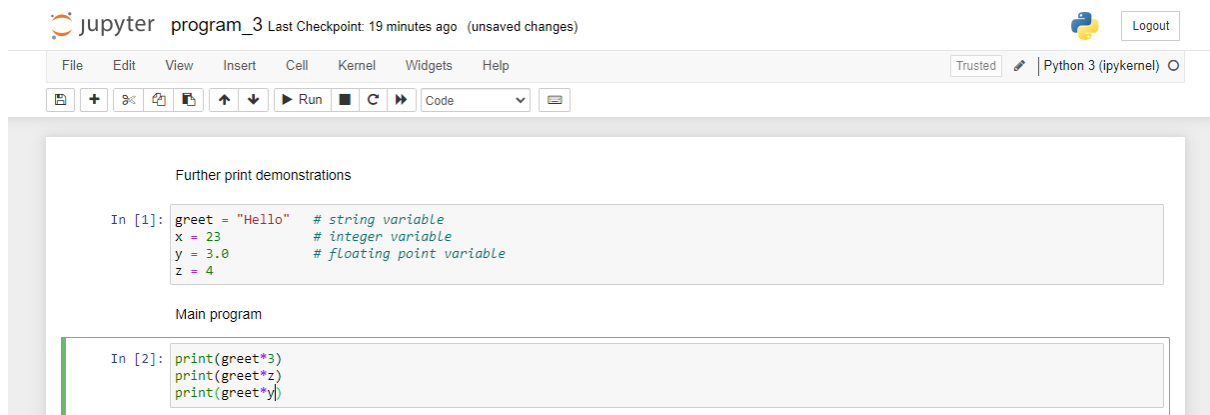
```
print(greet, end="")
print(name)
```

Further extend the program by adding these print statements to the bottom of the file:

```
print(x*y)                # multiply two numbers
print(greet*3)            # multi print a string
print("23 divide by 3.0 = ", x/y)    # divide two numbers
print("23 integer divide by 3.0 = ", x//y) # integer division
```

Notebook Practice: program_3.ipynb

Save a copy of *program_2.ipynb* as *program_3.ipynb*. Modify program_3.ipynb to the following:



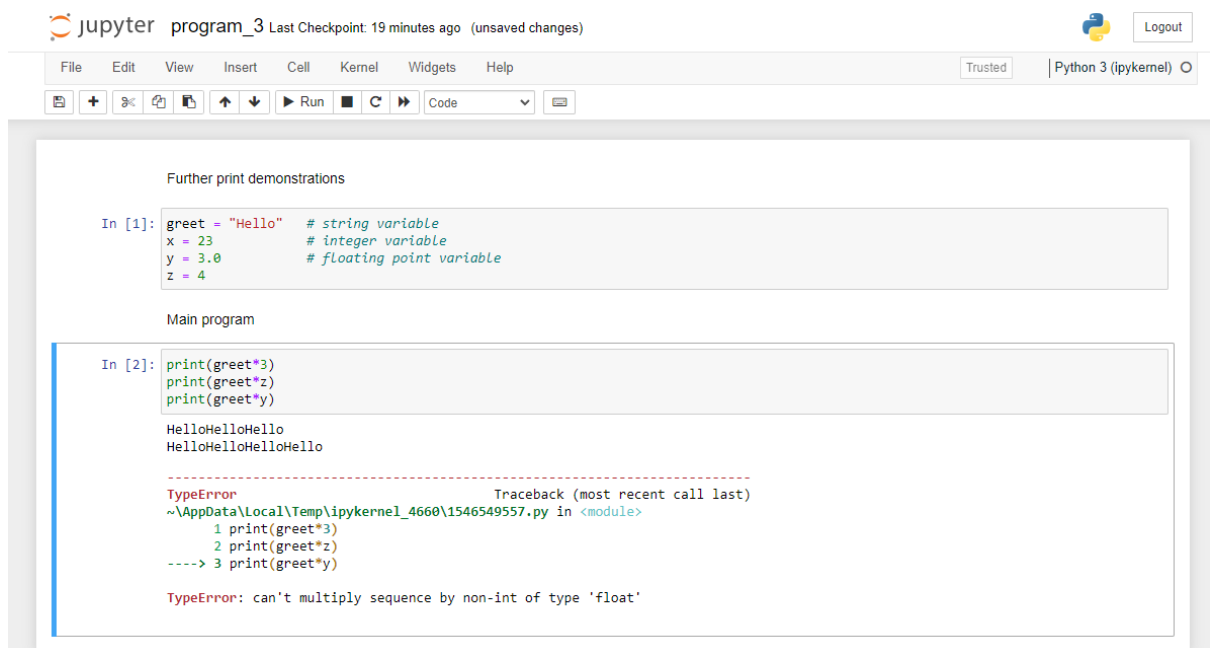
The image shows a Jupyter Notebook interface for a file named 'program_3'. The top bar indicates 'Last Checkpoint: 19 minutes ago (unsaved changes)' and includes a 'Logout' button. The menu bar contains 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. The toolbar shows icons for file operations, running, and code execution. The notebook content is divided into two sections: 'Further print demonstrations' and 'Main program'. The 'Further print demonstrations' section contains a code cell with the following code:

```
In [1]: greet = "Hello" # string variable
x = 23 # integer variable
y = 3.0 # floating point variable
z = 4
```

The 'Main program' section contains a code cell with the following code:

```
In [2]: print(greet*3)
print(greet*z)
print(greet*y)
```

Running the notebook will yield the following output:



The image shows the same Jupyter Notebook interface for 'program_3', but now with output and an error. The 'Further print demonstrations' section is the same as before. The 'Main program' section shows the output of the code cell:

```
HelloHelloHello
HelloHelloHelloHello
```

Below the output, a `TypeError` is shown with a traceback:

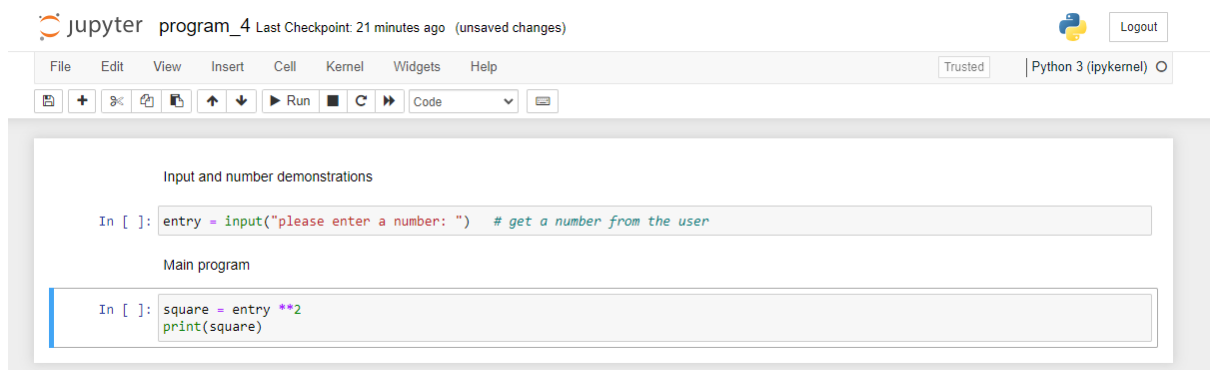
```
-----
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_4660\1546549557.py in <module>
      1 print(greet*3)
      2 print(greet*z)
----> 3 print(greet*y)

TypeError: can't multiply sequence by non-int of type 'float'
```

Which line of code caused the trace-back error? Why?

Notebook Practice: program_4.ipynb

Create a new program_4.ipynb notebook and type the following:



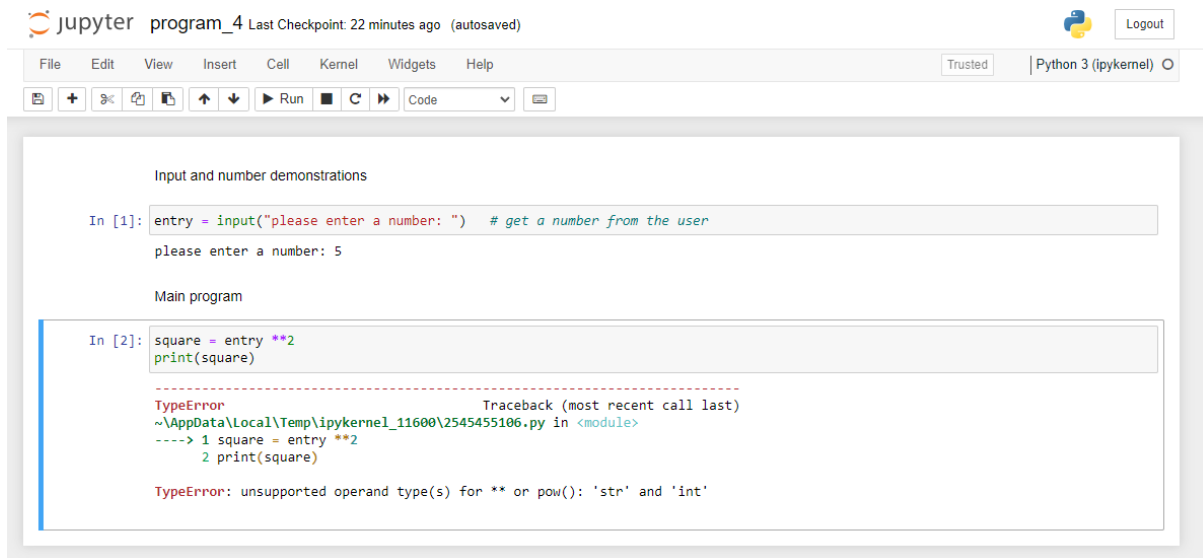
The image shows a Jupyter Notebook interface for a file named 'program_4'. The top bar indicates 'Last Checkpoint: 21 minutes ago (unsaved changes)' and includes a 'Logout' button. The menu bar contains 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. The toolbar shows icons for file operations, running, and code execution. The notebook content is divided into two sections: 'Input and number demonstrations' and 'Main program'. The 'Input and number demonstrations' section contains a code cell with the following code:

```
In [ ]: entry = input("please enter a number: ") # get a number from the user
```

The 'Main program' section contains a code cell with the following code:

```
In [ ]: square = entry **2
print(square)
```

Run your notebook and you should receive the following output:



The image shows a Jupyter Notebook interface with the title 'program_4' and a status 'Last Checkpoint: 22 minutes ago (autosaved)'. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and code execution. The notebook contains two code cells. The first cell, labeled 'In [1]:', contains the following code:

```
entry = input("please enter a number: ") # get a number from the user
```

 The output of this cell is 'please enter a number: 5'. The second cell, labeled 'In [2]:', contains the following code:

```
square = entry **2
print(square)
```

 The output of this cell is a `TypeError` message: `TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'`. The error message is displayed in red text, indicating a runtime error.

Why has your program failed?

Explanation

When using the `input()` statement to acquire data from a user, Python treats the `input()` literally as a set of characters (a character string) from the keyboard.

This can be corrected by *converting* the type from a string to an integer by casting.

Amend your code as shown below and try again:



The image shows a Jupyter Notebook interface with the title 'program_4' and a status 'Last Checkpoint: 23 minutes ago (unsaved changes)'. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and code execution. The notebook contains one code cell, labeled 'In [1]:', which contains the following code:

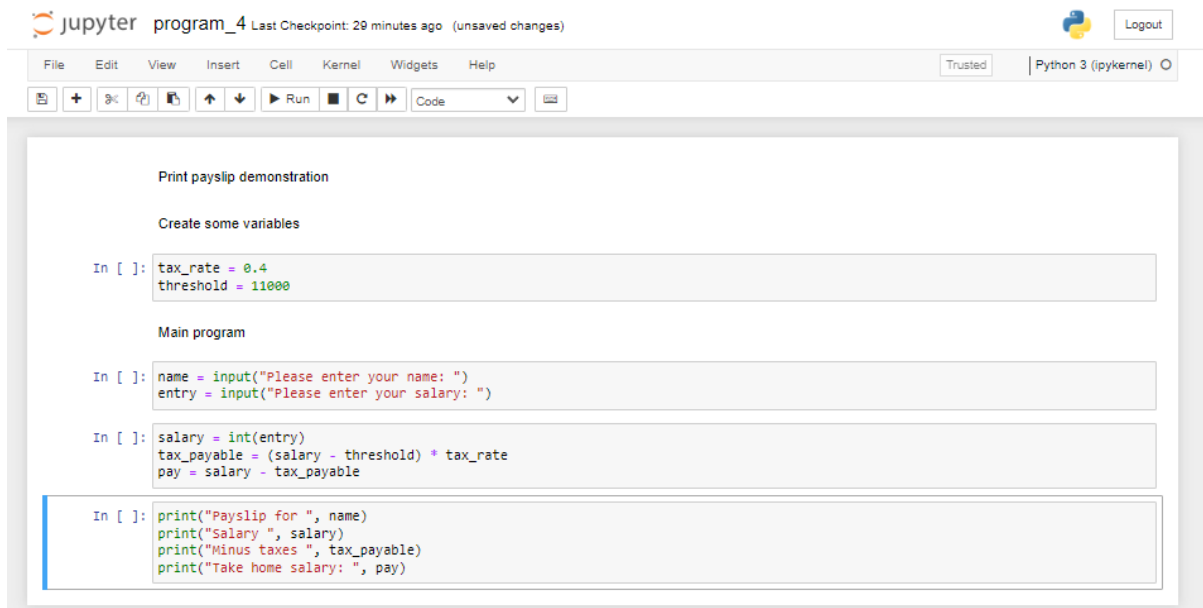
```
entry = input("please enter a number: ") # get a number from the user
num = int(entry)
square = num**2
print(square)
```

 The output of this cell is 'please enter a number: 5' followed by '25' on a new line, indicating that the program now runs successfully.

What did we do to correct the problem?

Pay slip Program: payslip.ipynb

Now for a more complex example. Create a new notebook and save as *payslip.ipynb*.



The screenshot shows a Jupyter Notebook titled 'program_4' with a last checkpoint 29 minutes ago. The notebook contains the following code:

```
Print payslip demonstration

Create some variables

In [ ]: tax_rate = 0.4
        threshold = 11000

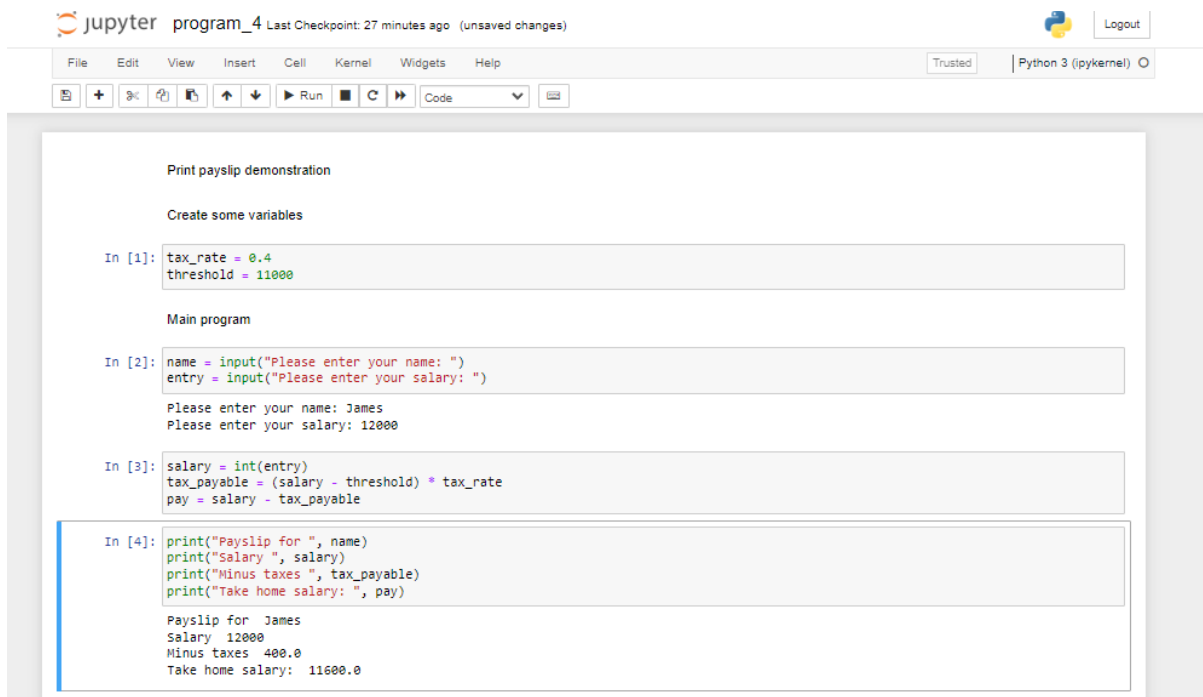
Main program

In [ ]: name = input("Please enter your name: ")
        entry = input("Please enter your salary: ")

In [ ]: salary = int(entry)
        tax_payable = (salary - threshold) * tax_rate
        pay = salary - tax_payable

In [ ]: print("Payslip for ", name)
        print("Salary ", salary)
        print("Minus taxes ", tax_payable)
        print("Take home salary: ", pay)
```

Save and Run the notebook using an initial salary of 12000.



The screenshot shows the same Jupyter Notebook after execution. The code is now numbered, and the output is visible for the last cell.

```
Print payslip demonstration

Create some variables

In [1]: tax_rate = 0.4
        threshold = 11000

Main program

In [2]: name = input("Please enter your name: ")
        entry = input("Please enter your salary: ")
        Please enter your name: James
        Please enter your salary: 12000

In [3]: salary = int(entry)
        tax_payable = (salary - threshold) * tax_rate
        pay = salary - tax_payable

In [4]: print("Payslip for ", name)
        print("Salary ", salary)
        print("Minus taxes ", tax_payable)
        print("Take home salary: ", pay)

Payslip for James
Salary 12000
Minus taxes 400.0
Take home salary: 11600.0
```

Sum and Product: sum_and_product.ipynb

Create a notebook to input three integers and print out their sum and product as illustrated below (N.B. The numbers 3, 4, 5 are just examples of possible input integers):

Integer 1: 3

Integer 2: 4
Integer 3: 5
The sum is 12 and the product is 60
Done!

Area of a Circle: circlearea.ipynb

Write a program to calculate and output the area of a circle.

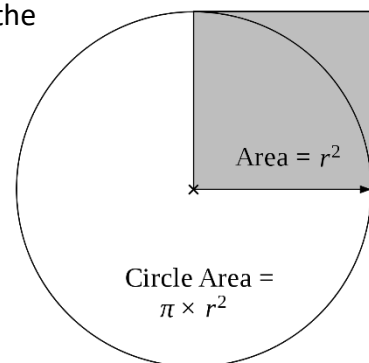
Hint: Python comes with the value of the mathematical constant pi (π) built in as `math.pi`. You'll be making heavy use of this over the next set of tasks. Remember, you need to add `import math` at the top of your file to be able to use this!

1. Create a new notebook file called "circlearea.ipynb".
2. Write Python code in your notebook to ask the user for the radius of a circle. The program will then calculate, store and output the area of the circle to the user. You can find the formula for the area of a circle here:

<http://www.calculateme.com/cArea/AreaOfCircle.htm>

3. Modify the program to also store and output the circumference (perimeter) of the circle. You can find the formula for this here:

<https://www.mathplanet.com/education/pre-algebra/more-about-equation-and-inequalities/calculating-the-circumference-of-a-circle>



Square Footage of a House

Write a program to calculate and output the square footage of a house.

1. Write a program that asks the user for the width and length of 4 rooms of a house, you can assume that the rooms are rectangular (i.e. no alcoves, etc.). Once all the information has been collected, the program will then display the square footage per room and then the total square footage for the entire house.
2. Copy your solution and modify it to also ask the height of each room so that it can calculate the volume for each room, and the entire house.
3. Use the usual $\text{width} \times \text{length}$ formula for calculating the area of a rectangle and $\text{width} \times \text{length} \times \text{height}$ for calculating the volume of a cuboid.

Celsius-Fahrenheit Conversion

Write a program that prompts the user to input a temperature in Celsius. The program will then output the temperature in Fahrenheit. The format of the output is required to be:

22 Celsius is 71.6 degrees Fahrenheit.

Get the formula for the conversion here:

<http://www.calculateme.com/cTemperature/CelsiusToFahrenheit.htm>